

Linear Data Structure

UNIT - 2

Linear Data Structure

- *“A Data Structure in which all elements are arranged sequentially in terms of Memory Blocks called Linear Data Structure”.*
- There are mainly four Linear Data Structure
 - Array
 - Stack
 - Queue
 - Linked List

1. ARRAY

WHAT IS ARRAY

- “Array is sequential collection of Data elements of same type”
- When array is used, sequential memory blocks are allocated to store values.

DECLARATION

- SYNTAX: *datatype variable_name[size];*
- For ex:
`int number[10];`
`char name[30];`

ACCESS ARRAY ELEMENTS

- ‘INDEX’ variable is used to access elements in array
- Syntax: *variable_name [index];*
- For Ex:
`for( i=0; i<10; i++)`
`printf(“element is %d”, ar [ i ]);`

1. ARRAY

INITIALIZATION OF ARRAY VARIABLE

- Initialization means allocate value to array variable at the time of declaration
- Two types of Initialization: Static and Dynamic

1. Static Initialization

- Syntax: *datatype variable_name [size] = { value list};*
- ‘value list’ type is same as ‘datatype’ and all values must be separated by coma specially in numeric typed array.
- For Ex:

```
int ar[3] = {4,7,2};
```

```
char name[10] = {'a' , 'e', 'i', 'o' };
```

```
char name[10] = "aeio"; //same as above
```

1. ARRAY

- So, in Initialization
 - Numerical values don't need single quote
 - Character values requires Single Quote (character wise) or Double quote (String wise)

2. Dynamic Initialization

- In which, values are taken from user at the time of execution using loop structure, one by one for each index in array.
- For Ex: for (i=0; i<10;i++)
 scanf("%d", &ar[i]);

1. ARRAY

PASS ARRAY AS AN ARGUMENT TO FUNCTION

- We can pass array to any User Defined function as an argument.

- For One-Dimensional Array

```
void fun_name ( int ar[], int size) {  
    //body of function  
}
```

- For Char typed array

```
void fun_name (char ar[]) {  
    //body of function  
}
```

- For Two-dimensional array

```
void fun_name (char ar[ ] [10] , int sR, int sC) {  
    //body of function  
}
```

1. ARRAY

POINTER AND ARRAY

- “POINTER is special variable which stores address of another variable”
- Address of array variable can be also stored in pointer variable.
- For ex:

```
int ar[10];
```

```
int *p;
```

```
p = ar;           // or   p = &ar[0]; (both are same)
```

- Means, array name itself represent starting address of array or first element address represent address of complete array.
- We can also access elements by using Pointer variable which stores address of array
- For Ex:

```
int *ptr = ar;
ptr++;
printf("%d", *ptr);           //prints value of 2nd element
```

1. ARRAY

- Through pointer variable, array traversal also can be possible.
- In which two things matters
 1. Data type of Pointer variable
 2. Increment Value to access next possible element in array
- **pointer_variable* will help us to access value at address which is referenced by *pointer_variable*
- So, to access next element,
$$next_element_address = Current_element_address + (size_of_datatype * increment_value)$$
- For ex:

```
int ar[5] = {3,4,5,6,9};
int *ptr = ar;
for ( i=0 ; i<5 ; i+=1 , ptr = ptr + 1 )
    printf("%d", *ptr);    //prints complete array
```

1. ARRAY

POINTER TO ARRAY

- We can also declare pointer variable that can refer array without help of array variable.
- That concept is known as “Pointer to Array”
- Syntax: `int *ptr_var[size];`
- For ex: `int *ptr[5];` // OR `int (*ptr[5]);` both are same
it stores 5 integer type values and starting address is stored in ‘ptr’.

ARRAY OF POINTER

- It is also possible to declare pointer array, which stores different addresses at each location in array. (collection of pointers)
- That concept is known as “Array of Pointer”
- Syntax: `int* (ptr_var[size]);`
- For ex: `int* (ptr[5]);` // `int* ptr[5];` both are same
- ptr is array of 5 pointers to type integer.

1. ARRAY

SPARSE MATRIX

- “Sparse matrix is a 2D array in which many elements can have value zero”
- For efficient utilization of memory, we can use Sparse Structure
- Basically Three types of Sparse Matrix:

1. Lower Triangle Sparse Matrix:

- All the elements above main diagonal line in matrix have value zero.
- $A[i][j] = 0$ for all $i < j$ in $n \times n$ matrix

2. Upper Triangle Sparse Matrix:

- All the elements below main diagonal line in matrix have value zero
- $A[i][j] = 0$ for all $i > j$ in $n \times n$ matrix

3. Tri-Diagonal Sparse Matrix:

- In which a matrix has non-zero values on main diagonal line and exact above and below diagonal lines of main diagonal line.
- $A[i][j] = 0$ for all $(i-j) > 1$ in $n \times n$ matrix

1. ARRAY

$$\begin{bmatrix} 1 & & & & \\ 5 & 3 & & & \\ 2 & 7 & -1 & & \\ 3 & 1 & 4 & 2 & \\ -9 & 2 & -8 & 1 & 7 \end{bmatrix}$$

Figure 3.34 Lower-triangular matrix

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ & 3 & 6 & 7 & 8 \\ & & -1 & 9 & 1 \\ & & & 9 & 2 \\ & & & & 7 \end{bmatrix}$$

Figure 3.35 Upper-triangular matrix

$$\begin{bmatrix} 4 & 1 & & & & \\ 5 & 1 & 2 & & & \\ & 9 & 3 & 1 & & \\ & & 4 & 2 & 2 & \\ & & & 5 & 1 & 9 \\ & & & & 8 & 7 \end{bmatrix}$$

Figure 3.36 Tri-diagonal matrix

- To store 2D matrix in 1D array, we can use Mapping Techniques.
- Two Mapping techniques mainly
 - **Row wise mapping**: Read values Row by row from matrix
 - **Column wise mapping**: Read values Column by column from matrix
- So, here for First Matrix mapping is
Row wise is: { 1, 5, 3, 2, 7, -1, 3, 1, 4, 2, -9, 2, -8, 1, 7}
Column wise is : { 1, 5, 2, 3, -9, 3, 7, 1, 2, -1, 4, -8, 2, 1, 7 }

1. ARRAY

APPLICATIONS OF ARRAY

- Arrays are widely used to implement mathematical vectors, matrices, and other kinds of rectangular tables.
- Many databases include one-dimensional arrays whose elements are records.
- Arrays are also used to implement other data structures such as strings, stacks, queues, heaps, and hash tables.
- Arrays can be used for sorting elements in ascending or descending order.

2. STACK

WHAT IS STACK?

- Stack is Linear Data Structure in which Element which is inserted LAST , is retrieved FIRST
- It's also known as "LIFO" [Last IN First OUT]
- For example, consider Pile of Plates, in which we put a plate at the top of it's pile on another plate.
- And we remove only topmost plate which was placed last.
- So, we can Add / Remove element from one position that is topmost position.
- Stack can be implemented using Array or Linked List.
- On which end, we can insert/remove element that end is known as TOP.
- So, TOP is a tracker which tracks the position in Stack for insertion or deletion of element.

2. STACK

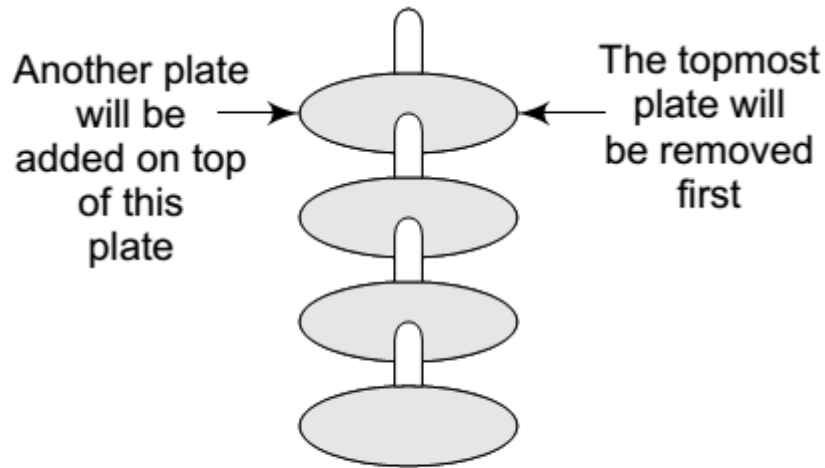


Figure 7.1 Stack of plates

ARRAY REPRESENTATION OF STACK

1	2	3	4	5					
0	1	2	3	TOP = 4	5	6	7	8	9

- TOP is tracker for element which is last inserted or removed
- If TOP = -1 then Stack is empty.
- In above example, Array[10] is as Stack and TOP=4, so total 5 elements are in stack.

2. STACK

OPERATIONS OF STACK

- There are basic three operations:
 - PUSH() - to insert element into stack at TOP most position
 - POP() - to remove element from stack from TOP most position
 - PEEP() - to retrieve value of TOP most element

1. PUSH()

- To insert element at TOP position in stack
- Before inserting element, check value of TOP, is it equal to size of array or not? If it's equal to size of array means $TOP == MAX$ then Stack is full, we can't insert an element into stack.
- If $TOP \neq MAX$ then first increment TOP by 1 and then insert element on TOP position.

2. STACK

Algorithm: PUSH (A, TOP, VAL)

1. Check for Overflow
 if $TOP == N-1$ then
 Print "Stack is Overflow"
 exit
 end if
2. Increment TOP
 $TOP = TOP + 1$
3. Insert Element
 $A[TOP] = VAL$
4. End

2. STACK

2. POP()

- To remove element from TOP position
- Before removing element, check $TOP == -1$ then stack is empty.
- Otherwise decrease TOP by 1 just.

Algorithm: POP (A, TOP)

1. Check for Underflow
if $TOP == -1$ then
 Print "Stack is Underflow"
 exit
end if
2. Fetch Value at TOP Position
 Print "Value at A[TOP] is deleted"
3. Delete Element
 $TOP = TOP - 1$
4. End

2. STACK

3. PEEP()

- To Retrieve value of element from TOP position
- Before removing element, check $TOP == -1$ then stack is empty.
- Otherwise get value at TOP position. (TOP remains as it is)

Algorithm: PEEP (A, TOP)

1. Check for Underflow
if $TOP == -1$ then
 Print "Stack is Underflow"
 exit
end if
2. Take Value from TOP Position
 $Value = A[TOP]$
3. Return Value
 Return (Value)
4. End

2. STACK

APPLICATIONS OF STACK

1. Reversing a list
2. Parentheses checker
3. Conversion of an infix expression into a postfix expression
4. Evaluation of a postfix expression
5. Conversion of an infix expression into a prefix expression
6. Evaluation of a prefix expression
7. Recursion
8. Tower of Hanoi